
Projeto: Análise Global de Mudanças Climáticas e Eventos Extremos (Spark Big Data)

1. Enunciado do Problema

O objetivo é processar o conjunto de dados históricos de temperaturas globais para identificar tendências de aquecimento e anomalias térmicas por país e cidade ao longo dos últimos 250 anos. O projeto deve demonstrar a superioridade do **Spark RDD/DataFrames** em lidar com cálculos estatísticos complexos e transformações de dados em larga escala.

Perguntas que o projeto deve responder:

1. **Média Móvel de Temperatura:** Qual a evolução da temperatura média anual global por década?
2. **Anomalias Locais:** Quais foram os 10 anos mais quentes para cada continente nos últimos 50 anos?
3. **Cidades em Risco:** Quais cidades apresentaram o maior desvio padrão de temperatura (instabilidade climática) no último século?
4. **Correlação de Estações:** Existe correlação entre o aumento da temperatura mínima e da temperatura máxima em zonas tropicais? (Uso de bibliotecas MLlib ou agregação complexa).
5. **Qualidade de Dados:** Identificar e filtrar registros onde a incerteza da medição (*LandAverageTemperatureUncertainty*) é superior a 10% da média histórica.
6. **Correlação entre Emissões e Aquecimento (Join):** Existe uma correlação estatística (Pearson Correlation) entre o aumento das emissões de CO2 de um país e o aumento da sua temperatura média nos últimos 50 anos?
 - o *Desafio:* Cruzar duas bases de dados diferentes por chave de "País" e "Ano".
7. **Ranking de Aceleração Térmica (Window Functions):** Quais são os 10 países onde a temperatura média subiu mais rapidamente na última década em comparação com a década anterior?
8. **Previsão de Tendência (Spark MLlib):** Utilizando Regressão Linear simples, qual a temperatura prevista para uma determinada cidade/país nos próximos 5 anos, baseando-se no histórico dos últimos 20 anos?

2. O Dataset (Conjunto de Dados)

Utilizaremos o dataset **Climate Change: Earth Surface Temperature Data**, compilado pela Berkeley Earth.

- **Conteúdo:** Dados de temperatura de 1750 até o presente, segmentados por país, cidade e estado. Contém milhões de linhas com colunas de dados, temperatura média e incerteza.
- **Volume:** Aproximadamente centenas de MB a alguns GB (ideal para demonstrar a

velocidade de processamento em memória do Spark).

Onde obter:

- **Kaggle:** [Climate Change: Earth Surface Temperature Data](#)
 - Utilize o `GlobalLandTemperaturesByCity.csv`

Estrutura do arquivo CSV:

Coluna	Tipo	Descrição
dt	Date	Data da medição (formato YYYY-MM-DD).
AverageTemperature	Double	Temperatura média mensal em Celsius.
AverageTemperatureUncertainty	Double	A margem de erro da medição.
City / Country	String	Identificação geográfica.
Latitude / Longitude	String	Coordenadas (ex: 42.59N , 1.44E).

Limpeza e Ajustes Necessários (Data Cleaning)

Serão necessários os seguintes tratamentos:

A. Tratamento de Valores Nulos (NaN)

Muitas cidades antigas não têm registros em determinados meses.

- **Ação:** Remover linhas onde **AverageTemperature** é nulo. No PySpark:
`df.dropna(subset=["AverageTemperature"])`.

B. Conversão de Tipos e Datas

O Spark pode ler a data como *String*.

- **Ação:** Converter a coluna **dt** para o tipo **Timestamp** ou **Date** e extrair o **Ano** e o **Mês** em

colunas separadas para facilitar as agregações (**groupBy**).

C. Limpeza de Coordenadas

As coordenadas vêm com letras (N, S, E, W).

- **Ação:** Se forem usar mapas, precisarão remover as letras e converter para valores numéricos (Ex: **42.59N** vira **42.59**, **1.44W** vira **-1.44**).

D. Filtro de Incerteza

Algumas medições têm uma incerteza de 5°C, o que compromete a média.

- **Ação:** Filtrar registros onde **AverageTemperatureUncertainty** seja menor que um limite aceitável (ex: 1.5°C).

3. O Desafio da Segunda Base (CO2)

Para responder a Pergunta 6, os dados de CO2 geralmente estão por **País/Ano**.

- **Ajuste necessário:** O dataset de temperatura está por **Cidade/Mês**. Os alunos deverão primeiro agregar a temperatura por **Média Anual por País** no Spark para só então realizar o **join** com a base de CO2.

A integração com a base de dados de CO2 transforma o projeto de uma análise estatística simples em um estudo de **correlação climática**.

Aqui estão os detalhes técnicos para que os alunos possam trabalhar com esse segundo dataset:

3.1. Onde obter a segunda base de dados

Existem duas fontes principais recomendadas para esse projeto:

- **Kaggle (Recomendado):** [CO2 and Greenhouse Gas Emissions](#)
 - ou aqui => **Our World in Data (Oficial):** [GitHub - owid/co2-data](#) (Procure pelo arquivo **owid-co2-data.csv**).
 - **Vantagem:** Esta base é atualizada anualmente e é considerada o padrão ouro em relatórios internacionais.

3.2. Formato e Estrutura dos Dados

Diferente da base de temperatura (que é por cidade/dia), a base de CO2 é organizada por **País**

e Ano.

As colunas essenciais são:

Coluna	Tipo	Descrição
country	String	Nome do país ou região (ex: Brazil, China, World).
year	Integer	Ano da medição.
co2	Double	Emissões anuais de CO2 em milhões de toneladas.
co2_per_capita	Double	Emissões divididas pela população do país.
total_ghg	Double	Total de gases de efeito estufa (incluindo metano, etc).

3.3. Limpeza e Ajustes Específicos (Data Cleaning no Spark)

Esta base exige uma limpeza estratégica antes do `join`:

- Remoção de Agregados Globais:** O dataset contém linhas onde o "country" é "World", "Asia" ou "Europe". Se os alunos querem comparar países, devem filtrar essas linhas para não distorcer a média.
- Sincronização de Nomes:** Às vezes, a base de temperatura chama um país de "USA" e a de CO2 de "United States". Os alunos precisarão usar `regexp_replace` no Spark para padronizar os nomes antes de unir as tabelas.
- Tratamento de Dados Históricos:** As emissões de CO2 antes de 1850 são quase nulas em muitos países. Recomenda-se filtrar o mesmo período da base de temperatura (ex: **1900 a 2020**).
- Preenchimento de Lacunas (Imputation):** Em anos de guerra ou instabilidade, alguns países não reportaram dados. Os alunos podem optar por excluir essas linhas ou usar uma técnica de *Forward Fill* (repetir o valor do ano anterior) usando `Window Functions` do Spark.

Resumo do Fluxo de Dados (SUGESTÃO):

- Leitura:** CSV (Kaggle) $\rightarrow$ DataFrame Spark.
- Limpeza:** Remover nulos + Filtrar Anos (>1900) + Filtrar Incerteza.
- Enriquecimento:** Extrair Ano da data + Join com base de CO2.
- Processamento:** Agregações complexas (Média por década, correlações).

5. **Saída:** Parquet (para o HDFS) e CSV reduzido (para gráficos em Python).

Este processo de limpeza é 70% do trabalho de um Engenheiro de Dados e garante que as conclusões sejam baseadas em dados confiáveis

4. Conteúdo do Relatório de Entrega

A. Documentação Técnica (Relatório em PDF)

O relatório deve ser o "guia" de como o projeto foi construído.

- **Arquitetura do Cluster:** Descrição do ambiente (ex: Databricks, AWS EMR ou Docker Local). Especificar quantidade de memória dos *Executors* e número de *Cores*.
- **Metodologia de ETL:** Explicação detalhada de como a limpeza foi feita no Spark (remoção de nulos, filtros de data e padronização de nomes de países para o *Join*).
- **Evidências do Spark UI:** Prints da interface do Spark mostrando o **DAG (Grafo Acíclico Dirigido)** e o tempo de execução dos estágios (*Stages*) mais pesados.
- **Análise de Otimização:** Descrição de onde foi aplicado o `.cache()` ou `.persist()` e qual foi o ganho de tempo observado.
 - **Comparativo de Tempo:** Comparar o tempo de execução da primeira consulta (sem cache) versus a segunda consulta (com cache ativado).

B. Código-Fonte (Notebook ou Script .py)

- **Pipeline de Ingestão:** Código de leitura dos CSVs brutos (Kaggle/Berkeley) diretamente para DataFrames.
- **Bloco de Transformação:** Scripts comentados realizando os *Joins* e as agregações para as 8 perguntas.
- **Uso de Spark SQL ou DSL:** Os alunos podem optar por usar `spark.sql("SELECT...")` ou funções de DataFrame (`df.select().groupBy()`).

C. Resultados (Visualização)

Como o Spark reduz bilhões de linhas para algumas dezenas de linhas de resultado, os alunos devem apresentar esses dados de forma legível.

- **Gráficos:** Pelo menos um gráfico de linha (ex. curva de aquecimento global baseada nos dados processados) e um gráfico de dispersão (correlação CO2 vs Temp).
- **As respostas para cada uma das 8 perguntas, crie um tópico específico:** O que colocar:
 - A resposta direta (ex: "O coeficiente de correlação encontrado foi 0,87")
 - Uma visualização (gráfico ou tabela pequena) que resuma o resultado.

- Uma breve interpretação dos dados (ex: "Observou-se que o aumento da temperatura em países da Europa foi mais acelerado que na América do Sul entre 1990 e 2020").
- Indicar no código fonte (com comentário): o trecho de código utilizado para obter a resposta a uma pergunta.

D. Artefatos de Saída (Data Lake Simulado)

- ~~Arquivo Parquet: Uma amostra do dataset final (após o join e limpeza) salvo no formato colunar Parquet.~~
- **Dicionário de Dados Final:** Um breve texto explicando o que cada coluna do arquivo Parquet representa após o processamento.

DICAS

Como a limpeza ocorre no Spark

O processo é feito através de **Transformações** no DataFrame. O Spark não altera o arquivo original (imutabilidade); ele cria um novo fluxo de dados limpos a partir do bruto.

1. Filtragem de Registros Inválidos

Se o dataset de temperatura tem linhas sem valor (nulos), você executa um comando que instrui todos os computadores do cluster a ignorarem essas linhas.

- No **PySpark:** `df.dropna()` ou `df.filter(col("AverageTemperature").isNotNull())`.

2. Padronização de Tipos (Casting)

Muitas vezes o Spark lê tudo como texto (String). A limpeza envolve converter colunas para formatos numéricos ou de data para que você possa fazer cálculos.

- **Ação:** Converter a coluna `dt` de *String* para *DateTime*.

3. Tratamento de Outliers e Anomalias

Você pode usar lógica de programação para remover erros de sensores. Por exemplo, se uma temperatura em Nova York aparecer como **150°C**, o Spark pode filtrar isso usando regras de negócio definidas por você.

4. Normalização de Strings

Nomes de cidades podem vir com espaços extras ou letras maiúsculas/minúsculas divergentes (ex: "são paulo", "São Paulo ", "SAO PAULO"). O Spark limpa isso em massa para que a agregação funcione corretamente.

- **Funções:** `trim()`, `lower()`, `regexp_replace()`.
-

PySpark